FDE ACADEMY · LESSON 5 OF 9

Agentic Workflows

A graph that knows, for every step, what to do when it fails.

Raunak Sharan
Enterprise Agentic AI Engineering

git checkout lesson-05

OBJECTIVES

By the end of today you can

01

Express a workflow as a graph with one typed state

02

Keep an audit that nodes append to and never rewrite

03

Give each node a timeout, retries and a failure mode

04

Retry transient failures, and never retry a bug

05

Pause for a human with interrupt(), then resume

06

Know what a checkpoint may deserialize, and why

SCRIPT VS WORKFLOW

A script runs steps; a workflow handles them

Lesson 4: a function

- One step raises: the request is lost
- One step hangs: everyone waits
- Review means “put it on a queue”
- The audit is whatever got logged

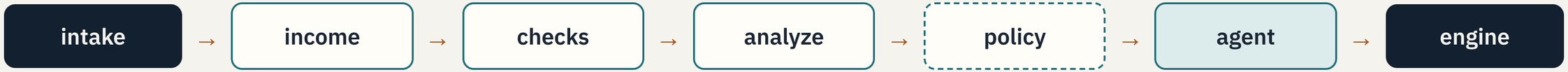
Lesson 5: a graph

- A failure becomes state, then an edge
- Every node has a timeout
- Review pauses the run and resumes it
- The audit is appended by a reducer

Same components, same contracts. Only the control flow moved, to workflow/graph.py.

THE GRAPH · UNDERWRITER GRAPH

Nine nodes, one way to a human



escalate

income, checks, analyze or engine failed: WORKFLOW_ERROR.

human_review

From escalate, or from engine on MANUAL_REVIEW. Queues, then pauses if interactive.

END

APPROVE and DECLINE go straight here from the engine.

TYPED STATE · WORKFLOW/GRAPH.PY

Nodes share one state, and history only grows

```
class UnderwritingState(TypedDict):
application: LoanApplication
checks: list[CheckResult]
case: UnderwritingCase
proposal: AgentRecommendation
decision: UnderwritingDecision
failure: dict | None
audit: Annotated[list[AuditEvent],
operator.add]
```

Return what changed

A node returns only the keys it set. LangGraph merges them.

Append, never replace

The reducer adds a node’s audit events. No node can rewrite history.

Failure is state

failure set → the next edge goes to escalate.

NODE POLICIES · NODE_POLICIES

Every node says how it fails

Node	Timeout	Attempts	On final failure
income	2 s	2	escalate · WORKFLOW_ERROR
checks	10 s	1	escalate (each check has its own status)
analyze	2 s	1	escalate
policy	3 s	2	degrade: no citations, logged
agent	60 s	1	fallback proposal; engine still decides
engine	1 s	1	escalate

One table in code, next to the graph. Changing a policy is a reviewed diff, not a hunt through try/except blocks.

RETRIES · WORKFLOW/RESILIENCE.PY

Retry the network. Never retry a bug.

Retried

Timeouts and TransientNodeError. The next attempt may succeed.

```
node_retry: succeeded on attempt 2
```

Not retried

ValueError, KeyError, RuntimeError. The same input fails the same way.

```
income failed after 1 attempt(s)
```

Every retry is in the audit. A retry you can’t see is a retry you can’t tune.

THE HUMAN-REVIEW EDGE

Pause the run, wait for a person, resume

```
pending = wf.start(app) # graph stops inside human_review
pending.request → {"review_case_id": "REV-APP-002",
"reason_codes": ["DTI_ABOVE_REVIEW_THRESHOLD"], …}
decision = wf.resume(pending.thread_id,
HumanReview(reviewer="u-1042", decision="APPROVE", note="…"))
```

interrupt()

Stops the node and saves state under the thread id.

HumanReview

Reviewer id, APPROVE or DECLINE, and a note. Validated.

Both on record

The engine’s MANUAL_REVIEW stays; final_recommendation is the human’s.

CHECKPOINTS

Resuming loads saved state, so allow-list it

What we saw

```
Deserializing unregistered type
underwriter.domain.models.LoanApplication from
checkpoint. This will be blocked in a future
version.
```

What we did

_serde() lists exactly our domain models and enums. Anything else in a checkpoint is refused.

We didn’t silence the warning.

Checkpoints are in memory today. A durable one (SQLite, Postgres) is exercise 1.

LIVE WALKTHROUGH

Break each node, then review a case

```
uv run underwriter graph
uv run underwriter run …/APP-001.json --node-fault income=transient
uv run underwriter run …/APP-001.json --node-fault income=error
uv run underwriter run …/APP-001.json --node-fault analyze=hang
uv run underwriter run …/APP-001.json --node-fault policy=error
uv run underwriter run …/APP-002.json --review-as APPROVE --reviewer u-1042
```

```
Reviewer: u-1042 decided APPROVE (REV-APP-002): Payroll deposits confirm income.
Final: APPROVE
```

LAB · 100 MINUTES

Three exercises

35 MIN · DURABILITY

SQLite checkpoints

Pass a SqliteSaver to build_graph and resume a review started by another process.

30 MIN · POLICY

Required or degrade?

Compliance wants no approval without a citation. Make it so, test first, then argue the other side.

35 MIN · OPERATIONS

Reviewer SLA

Add assigned_to and due_by to the pause, set by reason code from one table.

HOMEWORK AND INTERVIEW PREP

What strong work looks like

Criterion	Strong	Weak
State	Typed; audit by reducer	Dicts passed around
Node policy	Per node, written down	One big try/except
Retries	Transient only, audited	Everything, silently
Review	Pause, validate, record	Email and hope
Safety	Allow-listed checkpoints	Load anything

Interview questions

- Why a graph and not a function?
- Which failures do you retry?
- Human-in-the-loop, hours later
- Risks of loading a checkpoint
- Fail, degrade or fall back?

TAKEAWAY

Decide how every step fails before it does, and give the hard cases to a person.

Next · Lesson 6

Testing and evaluation

An eval harness over golden cases and injected failures, so every change is measured.